SQL Notes

Ques 1- What is SQl?
Ans- **Structured Query language**
.Sql is mainly used to communicate with Database.
.Sql is a mediator or translator between the user and database.

Ques 2-What is Database?
Ans- Database is a container in which the data is stored in a systematic & organized manner.

Ques 3-What is Data?
Ans- Data is a row facts that describe the attribute of an entity. .Row fact- Unchange information.
.Attribute-Properties
.Entity-Object

Ques 4-What is DBMS?
Ans-Database management system.
. It is mainly used to manage the database.

Characteristics:
1) Security:
(i)Username
(ii)Password
2) Authorization: e.g Otp

. In Dbms we store the data in the form of files.
. Data is stored with the help of Query language.

*** Types of DBMS:**

1)RDBMS
2)Object oriented DBMS
3)Hierarchical DBMS
4)Cloud

1)RDBMS-Relational Database management System
.Data store in the form of a table.
.Data store with the help of SQL.

Characteristics:
1) Security:
(i)Username
(ii)Password
2) Authorization: e.g Otp

***CRUD Operation:**

1)Create/Insert
2)Read/fetch
3)Update
4)Delete

Ques 5- Difference between DBMS and RDBMS ?

Ans	**DBMS**	**RDBMS**
1.	Data base management System.	Relational Database management System.
2.	It is to use the Query language.	It uses SQL.
3.	It stores the data in the form of files.	It stores the data in the form of a table.
4.	It will provide Security,authorization.	It also provides Security, authorization.
5.	CRUD operation is more difficult.	CRUD operation is easy.

***Relational model:**

Any DBMS is following the relational model called RDMS.
.Relational model states that the data should be stored in the form of a table.

Ques 6-**EF Codd Rules ?**

Ans- 1) The data entered into a cell must be a single value data.

NAME	VALUE
AMAN	8978653733
SAHIL	9348342023
RAVI	9974938443

2)We can insert everything into a column including metadata.
Like- integer,string,characters,image,binary character ,audio,video,files.

			META TABLE		
SNAME	AGE	IMAGE	IMGNAME	SIZE	FORMAT
RAVI	23	[0!]	IMG001	1MB	JPG
AMAN	30	[0!]	IMG002	2.5MB	JPEG
SAHIL	18	[0!]	IMG003	2MB	PNG

3)We can insert the data into multiple tables if necessary. We can establish a connection b/w these multiple tables with the help of key attributes.
i)Primary key
ii)Foreign key

4)The data entered into a column must be validate by two steps.
A)Data Type.
B)Constraints.

A) **Data type**-Data type is nothing but the different value to be inserted into a column.

Types of Data Type-
a)Char
b)varchar/varchar 2
c)Date
d)Number
e)Large Object{CLOB(CHARACTER LARGE OBJECT,BLOB(BINARY LARGE OBJECT)}

a)Char-It is use to store 'A to Z' ,'a to z','o'to '9' , special character -'!', '@','#','$','%','&"

alphanumeric value e,g email,password,username,domain,IP address,Pan card,IFSC name,Car number

syntax-colName Char(size)

size-It is nothing but total numbers of characters we can insert into a particular column.
Limitation- we can insert maximum 2000 characters. This Char is also called Fixed length memory allocation. E.X- Phone no,Ac no,OTP,Atm no.
char(10)
[S|A|H|I|L| | | | |]
used waste
Memory memory

b)Varchar-It is use to store 'A to Z' ,'a to z','o'to '9' , special character -'!', '@','#','$','%','&" ,alphanumeric value .
syntax- ColName varchar (size)
Size-maximum number of characters we can store in it known as Size. Limitation- we can insert maximum 2000 characters. This varchar is also called as variable length memory allocation. Varchar(10)
[S|A|H|I|L| | | | |]
used free
memory memory
Varchar 2 we can insert maximum 4000 characters.
.Internally varchar is consider as varchar 2

c)Date-It is mainly used to store the date in a specified format.
syntax- colName Date ;
e.g
ORacle specifies formate
1)'DD-MM-YY' ->'14-FEB-24' 2)'DD-MON-YYYY'->'14-FEB-2024'

d)Number-It is mainly used to insert an integer value into a column.
syntax-colName Number(precision[,scale])
scale is optional value,scale is for decimal
precision for integer

case1
(precision >scale)
e.g number(5)
5 is precision scale is 0
+-[9|9|9|9|9]
(-99999) to (99999) e.g 12345,34343
number(4,2) 4 is presion and 2 is scale.

+-[9|9|9|9] result is -99.99 to +99.99
Case 2
(precision=scale)
e.g number(5,5) it will create 5 memory blocks internally. +-[9|9|9|9|9]

result range will be -.99999 to +.99999

case 3
(precision<scale)
e.g number(2,6) it will create 6 memory blocks internally.

+-[0|0|0|0|9|9]
result range will be -.000099 to +.000099
PROCEDURE - SUBTRACT SCALE & PRECISION (6-2=4)
WE will insert zero from left hand side

e)LARGE OBJECT-it is mainly used to store large amounts of values.

i)CLOB(CHARACTER LARGE OBJECT)-it is used to store large amounts of characters upto 4GB.
e.g article ,book,subtitle

syntax- colName CLOB

ii)BLOB(BINARY LARGE OBJECT)-it is mainly used to store images,video,audio and file in binary form.
We can store maximum of 4GB of values.

syntax- colName BLOB

B) Constraints->
Constraints are nothing but the rules and regulations that need to be follow in order to insert the data into a particular column.

Types of Constraints ->

i)Unique Constraint
iiNot Null Constraint.
iii)Check Constraint.
iv)Primary key Constraint.
v)Foreign key Constraint.

i)Unique Constraint -> Unique is a Constraint mainly used to avoid the repeated value in a **column.**

syntax .

colName data type unique;

e.g - phone no,otp,aadhar no.

ii)Not Null Constraint-> Not null is a Constraint mainly used to avoid a null value in a particular column.

syntax .

colName datatype NOT NULL ;

e.g - registration no., emp id,name.

NULL ->NULL does not represent 0 or space ,any operation performed on null will result in null itself.

- ★ Null does not occupy any memory.
- ★ Null can not equate to null.

iii)Check Constraint -> Check is a Constraint which is mainly used along with a condition if the condition is satisfied then value will be inserted into the column.
It is used to provide an extra validation for the column .

syntax.

colName data type Check(condition) ;

e.g - age number(2) check (age>=18)
phone no number(10) check (phone.no >0)
Check(length(phone.no)=10) ;

iv)Primary key Constraint -> Primary key is mainly used to identify the records uniquecally in the table.

Characteristics -

- ★ Primary key will always accept unique values.
  - ■ Primary key follows unique Constraint.
  - ■ Primary key accepts Not Null values.
  - ■ Primary key follows NOT NULL Constraint.
  - ■ Primary key is a combination of unique and NOT NULL constraints.
  - ■ We can have one primary key in a table.

Syntax.

colName datatype primary key

v)Foreign key- Foreign key is mainly used to establish a connection between two or more tables.

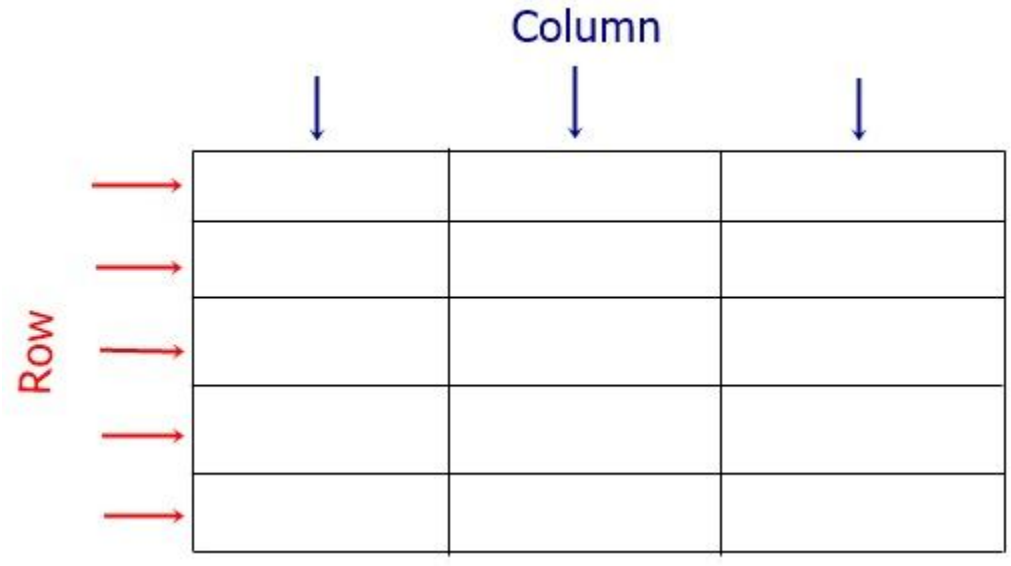
Characteristics-

- Foreign key can accept repeated or duplicate value
- Foreign key do not follow unique constraints.
- Foreign key accept NULL values.
- Foreign key is not follow not null Constraint.
- Foreign key is not a combination of unique and NOT NULL constraints.
- We can have multiple foreign key.
- We can called it Referential integrity Constraint.
- To use a column name as a foreign key it should be a primary key in its own table.

syntax-

colName datatype foreign key;

***Column**: Columns are the vertical line in which we can store the properties /attributes.
***Rows**: Rows are the horizontal line ,they are also known as tuples and records.
***Cell**:Cell is the intersection of rows and columns. .Cell is a smallest unit of a table.



Ques 7- **Types of SQl Statement:**

Ans- **5 types of SQl Statement:**

1.DQL-Data Query Language.
It is used to read or fetch the record from a database or table.

Sub statement->Select ,Projection, Selection, Joins.

2.DDL- Data Definition language.
It is used to create,Update,Delete the **table**.
Sub statement->Create, Alter, Rename, Drop, Truncate.

3.DML-Data Manipulation Language.
It is used to insert,update or delete the **record.**
Sub statement->Insert, Update, Delete.

4.DCL-Data Control Language.
It is used to control the access over a database.
Sub statement->Grant, Revoke.

5.TCL-Transaction Control Language
.It is used to control the transection of data.
Sub statement->Commit, Savepoint ,Rollback.

1.DQL-Data Query Language

DQL is used to fetch the data from the table.

In DQL ,we have 4 statements:
Select ,Projection, Selection, Joins.

1)Select clause: Select is used to select the data and display it on the output screen.
2)Projection:Projection is used to retrieve the data by selecting only the columns.
3)Selection: Selection is used to retrieve the data by selecting both rows and columns.
4)Joins: Joins are used to retrieve the data from multiple tables.

Projection Syntax

Syntax:Way of writing a query.

```
Select */[Distinct]ColName/Expression[Alias]
From TableName;
```

*****-use to display all the details of the table.
from-from clause will go into the database and search the table Name and it will be put on under execution.

Order of Execution - 1) from clause
2) select clause

Expression-It is the statement that gives you some result.

Operators-Operators are nothing but the mathematical symbols.

Operands-Operands are nothing but the values.

ex. 5+6

here 5,6 are the operands and + is the operator.

Ques 8- **What is Distinct?**
Ans- It is used to remove(avoid) the repeated value from the **result table**.

Characteristics of distinct

1) District is always passed as the first argument inside the select clause.
2) If we have multiple columns in distinct clauses then it will remove combinations of values from the columns in which records are repeating.

Syntax-

```
SELECT DISTINCT COLNAME
FROM TABLENAME ;
```

.WAQTD unique branch of a student.

```
SELECT DISTINCT BRANCH
FROM STUDENT;
```

**** TO display all the columns with a query.

```
syntax- SELECT EMP.* SAL *12
                FROM EMP ;
```

**** WHEN wrong query write or to edit the query.

syntax- ED

**** TO save the data/table

.. First create a folder ,then copy the folder path. syntax- SPOOL "FOLDER PATH\FILE NAME" ;

..TO end the spool
syntax- SPOOL OFF ;

**** TO set the pages and lines
syntax-SET PAGES 200 LINES 200 ;

Ques 9- **What is Alias ?**
Ans- It is nothing but just an alternative name given to a temporary column.

.We can give an Alias with or without a keyword AS 4 Ways to write

Query with Alias->

1) SELECT SAL*12 AS "ANNUAL SALARY"
FROM EMP;

2) SELECT SAL*12 AS ANNUAL_SALARY
FROM EMP;

3) SELECT SAL*12 AS ANNUAL_SALARY
FROM EMP;

4) SELECT SAL*12 "ANNUAL_SALARY"
FROM EMP;

***

20/08/2024

Selection: in this we have some condition (where) it is used to retrieve the data by selecting only rows and columns.
Syntax

Select colName/Expression
from tableName
where <filter condition>;

filter condition is like a **conditional statement.**

e.g - Query:

Select * (3)
from emp (1)
where deptno=20 ; (2) the order of execution Order of execution:

from -from clause will execute (row by row)

Where-where clause will execute (row by row)
Select-select clause will execute (row by row)

Emp table

Empno	ename	Job	sal	deptno.
1.	Rohit	Clerk	2000	10
2.	Sumit	MANAGER	3000	15
3.	Abhishek	SD	2000	10
4.	Ravi	WD	5000	20
5.	Rahul	WD	5000	20

Output:

Empno ename Job sal deptno. 4. Ravi WD 5000 20 5. Rahul WD 5000 20

* WAQTD the details of employee who are working as a manager 9

```
Select *
 from emp
where job='MANAGER' ;
```

Output:

Empno ename Job sal deptno.
2. Sumit MANAGER 3000 15

21/08/2024

Ques 10- **Operators ?**
Ans- 1) Arithmetic operators(+,-,*,/)
2) Concatenation operator(||)
3) Comparison operator(=,!=)
4) Relational operator (>,<,>=,<=)
5) Logical operator (AND,OR,NOT)
6) Special operator
1)IN
2)Not In
3)Is

4)Is Not
5)Between
6)Not Between
7)Like
8)Not Like

7)SubQuery Operator(ALL,ANY,EXISTS,NOT EXISTS)

2.Concatenation operator

concatenation operator is used to combine two or more strings into one string

Symbol-||

.WAQTD the string 'harshal''Rule' on the output screen?

```
SELECT 'harshal'||'Rule'
FROM DUAL ;
Output-harshalRule
```

.WAQTD the string like 'Mr Smith' for each employee?

```
SELECT 'Mr ' || ENAME
FROM EMP ;
```

2.Logical operator (AND,OR,NOT)-

AND operator- if all conditions are satisfied then only we will get an output.

I/P	I/P	O/P
1	1	1
0	1	0
1	0	1
0	0	0

OR operator- if any one condition is satisfied then we will get an output.
NOT operator- if condition is True->False
if condition is False->True

22/08/2024

**WAQTD details of an employee if an employee is working in department 20 and his salary is greater than 2500

SELECT *

FROM EMP
WHERE DEPTNO=20 AND SAL >2500 ;

**WAQTD EMP NAME,SAL,DEPT NO,OF AN EMPLOYEE WHO ARE HIRE DATE AFTER 83 AND SAL IS LESS THAN 4000

SELECT ENAME,SAL,DEPTNO
FROM EMP
WHERE HIREDATE>'01-JAN-83' AND SAL>4000;

When we use > and < then we use AND OPERATOR.

23/08/2024

6) Special operator

1) IN OPERATOR

- It is mainly used to handle multiple values on the right hand side.
- If any one value is satisfying the condition then the IN operator will give you that record. (IN IS LIKE =)
- It will work like an **OR operator.**

Syntax-

ColumnsName IN('v1','v2','v3'....n)

WAQTD ENAME,JOB,WHO ARE WORKING AS MANAGER ,ANALYST AND SALESMAN.

SELECT ENAME,JOB
FROM EMP
WHERE JOB IN ('MANAGER','ANALYST','SALESMAN') ;

2) NOT IN OPERATOR

- It is mainly used to handle multiple values on the right hand side
- Instead of selecting the values it will **reject those values.**

Syntax-

ColName NOTIN('v1','v2,'v3',.....'n')

WAQTD THE DETAILS OF EMPLOYEE WHO ARE NOT WORKING IN DEPT 10,20

SQL> SELECT ENAME
2 FROM EMP

3 WHERE DEPTNO NOT IN (10,20) ;

WAQTD ALL THE EMPLOYEE NAME AND JOB OF AN EMP EXCEPT THE MANAGER SQL> SELECT ENAME,JOB
2 FROM EMP
3 WHERE JOB NOT IN 'MANAGER' ;

3) IS OPERATOR

- It is mainly used to compare the NULL values.

Syntax-

ColName IS NULL ;

WAQTD DETAILS OF THE EMPLOYEE WHO ARE NOT EARNING ANY COMMISION

SQL> SELECT *
2 FROM EMP
3 WHERE COMM IS NULL ;

WAQTD DETAILS OF THE EMPLOYEE WHO DON'T HAVE ANY MANAGER ; SQL> SELECT *
2 FROM EMP
3 WHERE MGR IS NULL ;

26/08/2024

4)IS NOT OPERATOR

- Is mainly used to compare non-NULL values.

Syntax-

ColName IS NOT NULL;

WAQTD NAME OF AN EMPLOYEE WHO ARE EARNING ANY COMM

SQL>
1 SELECT ENAME
2 FROM EMP
3* WHERE COMM IS NOT NULL

5)Between OPERATOR

- It is mainly used to obtain the values between the range of lower value and higher value.
- It will accept the ranges of value as well.

Syntax-

ColName BETWEEN LOWER VALUE AND HIGHER VALUE ;

CASE 1-
WAQTD THE ENAME AND SAL OF AN EMPLOYEE IF THE EMPLOYEE IS EARNING

IN THE RANGE OF 2000 AND 3000 ?

```
SQL> SELECT ENAME,SAL
 2 FROM EMP
 3 WHERE SAL BETWEEN 2000 AND 3000 ;
```

CASE 2-
WAQTD THE ENAME AND SAL OF AN EMPLOYEE IF SAL GREATER THAN 2000 AND LESS THAN 3000 ?

```
SQL> SELECT ENAME,SAL
 2 FROM EMP
 3 WHERE SAL BETWEEN 2001 AND 2999 ;
```

CASE 3-
WAQTD THE ENAME AND SAL OF AN EMPLOYEE IF EMP IS EARNING BETWEEN 2000 AND 3000?

```
SQL> SELECT ENAME,SAL
 2 FROM EMP
 3 WHERE SAL BETWEEN 2001 AND 2999 ;
```

WAQTD DETAILS OF AN EMPLOYEE WHO ARE HIRED IN 87 WITH THE HELP OF BETWEEN OPERATOR

```
SQL>
 1 SELECT *
 2 FROM EMP
 3 WHERE HIREDATE BETWEEN '01-JAN-87' AND '31-DEC-87' ;
```

WAQTD DETAILS OF AN EMPLOYEE WHO ARE HIRED RANGE 82 TO 87

```
SQL> SELECT *
 2 FROM EMP
 3 WHERE HIREDATE BETWEEN '01-JAN-82' AND '31-DEC-87' ;
```

6) NOT BETWEEN

- This is the same as the **Between operator.**
- Instead of selecting the value it will **reject the value.**

WAQTD THE DETAILS OF AN EMPLOYEE WHO ARE NOT EARNING IN THE RANGE OF 2000 AND 3000

```
SQL> SELECT *
```

```
 2 FROM EMP
 3 WHERE SAL NOT BETWEEN 2000 AND 3000 ;
```

WAQTD THE DETAILS OF AN EMPLOYEE WHO ARE EARNING LESS THAN 2000 AND GREATER THAN 3000 ?

```
SQL>
 1 SELECT *
 2 FROM EMP
 3 WHERE SAL NOT BETWEEN 2000 AND 3000;
```

WAQTD THE DETAILS OF AN EMPLOYEE WHO ARE HIRED BEFORE 82 AND HIRED AFTER 85?

```
SQL> SELECT *
 2 FROM EMP
 3 WHERE HIREDATE NOT BETWEEN '01-JAN-82' AND '31-DEC-85' ;
```

7)LIKE

- It is mainly used to match the pattern.
- % -this operator will accept multiple values or it will accept 0 values.
- _ -it will accept only one value.

Syntax-

```
colName LIKE 'pattern' ;
```

WAQTD NAME OF EMPLOYEE WHOSE NAME START WITH A?

```
SQL> SELECT ENAME
 2 FROM EMP
 3 WHERE ENAME LIKE 'A%' ;
```

WAQTD NAME OF EMPLOYEE WHOSE NAME END WITH T?

```
SQL>
 1 SELECT ENAME
 2 FROM EMP
 3* WHERE ENAME LIKE '%T'
```

WAQTD THE DETAILS OF AN EMPLOYEE WHOSE NAME CONSIST OF AA?

```
SQL>
 1 SELECT *
 2 FROM EMP
 3 WHERE ENAME LIKE '%A%A%';
```

27/08/2024

WAQTD DETAILS OF AN EMPLOYEE WHOSE NAME CONSIST OF CONSECUTIVE A? SQL> SELECT *
2 FROM EMP
3 WHERE ENAME LIKE '%AA%';

WAQTD NAME AND JOB OF EMPLOYEE WHOSE JOB CONSIST OF MAN AS A FIRST THREE CHARACTERS

SQL> SELECT ENAME,JOB
2 FROM EMP
3 WHERE JOB LIKE 'MAN%' ;

WAQTD NAME AND JOB OF EMPLOYEE WHOSE JOB CONSIST OF MAN AS A LAST THREE CHARACTERS

SQL> SELECT ENAME,JOB
2 FROM EMP
3 WHERE JOB LIKE '%MAN' ;

WAQTD NAME OF AN EMPLOYEE WHO ARE HIRED IN FEB

SQL> SELECT ENAME
2 FROM EMP
3 WHERE HIREDATE LIKE '%FEB%' ;

WAQTD DETAILS OF AN EMPLOYEE WHOSE NAME CONSIST OF A AS A SECOND CHARACTERS

SQL> SELECT *
2 FROM EMP
3 WHERE ENAME LIKE'_A%' ;

WAQTD DETAILS OF AN EMPLOYEE WHOSE NAME CONSIST OF E AS A SECOND LAST CHARACTERS
SQL> SELECT *
2 FROM EMP
3 WHERE ENAME LIKE '%E_' ;

WAQTD DETAILS OF EMPLOYEE IF AN EMPLOYEE IS EARNING 4 DIGIT OF SALARY

SQL> SELECT *

```
2 FROM EMP
3 WHERE SAL LIKE '____' ;
```

8) NOT LIKE

- ❖ It is similar to the **Like operator** but instead of selecting the value it will reject.

Syntax-

colName NOT LIKE ' PATTERN' ;

WAQTD DETAILS OF AN EMPLOYEE IF AN EMPLOYEE NAME IS NOT STARTING WITH S

```
SQL> SELECT *
 2 FROM EMP
 3 WHERE ENAME NOT LIKE 'S%' ;
```

QUES 11 -What is FUNCTION ?

ANS- Function is nothing but a block of code which is used to perform the specific task.

2 type of function

1)Pre defined function (also called a built-in function)
2)User defined function /store procedure

1)Pre defined function-> is a function that is already defined and provided by the programming language

i)Single row function

- ❖ Is a function which accepts multiple inputs and gives us multiple respective outputs.

ex-length(ename)

1 I/P ----------> 1 O/P

FUNCTION

2 I/P ----------> 2 O/P
FUNCTION
3 I/P ----------> 3 O/P
FUNCTION

ii)Multi row function/Group Function/Aggregate Function

- It is a function which accepts multiple inputs in a single shot and gives us one output.

Ex.

1 I/P ---------->
FUNCTION

2 I/P ----------> 1 O/P
FUNCTION
3 I/P ---------->
FUNCTION

5 types of multi row function

a)MAX()-it is used to obtain the maximum value present in the column.
b)MIN (): it is used to obtain the minimum value present in the column.
c)SUM(): it is used to obtain the summation of values present in the column.
d)AVG(): it is used to obtain the average of values present in the column.
e)COUNT(): it is used to obtain the number of values present in the column.

Characteristics of multi row function ->

1) Multi row function will accept only one argument.

E.g-

```
max(colName/Expression)
```

2)If we are passing a multi row function along with that we can **not pass any column name inside the select clause.**

e.g

```
select max(),Ename
```

3) Multi row function will ignore the **null values.**

4) We can not pass a multi row function inside the **where clause.**
5) Count is a only function in which we can pass * .

WAQTD THE MAXIMUM SAL OF AN EMPLOYEE
SQL> SELECT MAX(SAL)
2 FROM EMP;

WAQTD THE MAXIMUM SAL GIVEN TO A MANAGER
SQL>
1 SELECT MAX(SAL)
2 FROM EMP
3* WHERE JOB='MANAGER'

WAQTD MINIMUM SAL OF EMPLOYEE WHO ARE HIRED AFTER 81 AND BEFORE 87
SQL>

1 SELECT MIN(SAL)
2 FROM EMP
3* WHERE HIREDATE BETWEEN '01-JAN-82' AND '31-DEC-86'

WAQTD TOTAL SAL EARN BY EMP WHO ARE MAKING IN DEPT 10?
SQL>
1 SELECT SUM(SAL)
2 FROM EMP
3* WHERE DEPTNO='10'

WAQTD TOTAL NO OF EMP EARNING MORE THAN 1500 IN DEPT 10?
SQL> SELECT COUNT(ENAME)
2 FROM EMP
3 WHERE SAL>1500 AND DEPTNO IN 10 ;

WAQTD NO. OF EMPLOYEE WHO ARE GETTING COMM
SQL> SELECT COUNT(ENAME)
2 FROM EMP
3 WHERE COMM IS NOT NULL ;

WAQTD AVG SAL OF ALL THE EMPLOYEE WHOSE NAME STATED WITH S SQL> SELECT AVG(SAL)
2 FROM EMP
3 WHERE ENAME LIKE 'S%' ;

29/08/24

** Group By->

- ❖ It is mainly used to group similar types of records.

Syntax->

SELECT GROUP BY COLUMN/GROUP BY EXPRESSION (4)

FROM TABLENAME (1)

WHERE <FILTER CONDITION> (2)

GROUP BY COLNAME/EXPRESSION ; (3)

ORDER OF EXECUTION

1) FROM-ROW BY ROW
2) WHERE -ROW BY ROW
3) GROUP BY-ROW BY ROW
4) SELECT-GROUP BY ROW

Characteristics ->1)Group by clause will execute row by row.

2)After execution of group by clause you will get a group of

similar types of records.

3)After group by clause any clause will execute group by group.

4)The column name passes inside the group by clause and

also can pass inside the select clause along with a multirow

function known as **group by expression.**

WAQTD MAX SAL EARN BY AN EMPLOYEE IN EACH JOB ?

```
SQL> SELECT MAX(SAL),JOB
  2  FROM EMP
  3  GROUP BY JOB
```

****Having->**

- It is mainly used to filter the groups.

Syntax-

```
SELECT GROUP FUNCTION/GROUP BY EXPRESSION
FROM TABLENAME
WHERE <FILTER CONDITION>
GROUP BY COLNAME/EXPRESSION
HAVING <GROUP_FILTER_CONDITION> ;
```

ORDER OF EXECUTION

1)FROM(ROW BY ROW)

2)WHERE(ROW BY ROW)

3)GROUP BY(ROW BY ROW)

4)HAVING(GROUP BY GROUP)

5)SELECT(GROUP BY GROUP)

WAQTD TOTAL NO.OF EMPLOYEE WORKING IN EACH

DEPARTMENT IN WHICH MINIMUM TWO EMPLOYEE ARE

WORKING

```
SQL> SELECT COUNT(*) ,DEPTNO
  2  FROM EMP
  3  GROUP BY DEPTNO
  4  HAVING COUNT(*) >=2 ;
```

```
SQL> --WAQTD JOB OF EMPLOYEE IN WHICH MINIMUM 3
```

EMPLOYEE ARE WORKING --

```
SQL> SELECT JOB
  2  FROM EMP
  3  GROUP BY JOB
  4  HAVING COUNT(*)>=3 ;
```

```
SQL> -- WAQTD SALARY OF EMPLOYEE WHICH IS REPEATED--
```

```
SQL> SELECT SAL
  2  FROM EMP
  3  GROUP BY SAL
  4  HAVING COUNT(*) >=2 ;
SQL> --WAQTD NO OF EMPLOYEE WORKING IN EACH DEPT
HAVING AT LEAST TWO EMP WITH CHARACTER A OR S IN THEIR
NAME----


  1  SELECT COUNT(*),DEPTNO
  2  FROM EMP
  3  WHERE ENAME LIKE '%A%' OR ENAME LIKE '%S%'
  4  GROUP BY DEPTNO
  5* HAVING COUNT(*)>=2
```

****ORDER BY->**

- It is mainly used to arrange the records in **ascending[ASC]** or

descending[DESC] order in the Result table.

Syntax:

SELECT COLNAME/EXPRESSION

FROM TABLE NAME

WHERE (FILTER CONDITION) [OPTIONAL]

GROUP BY COLNAME

HAVING <GROUP_FILTER_CONDITION>

ORDER BY COLNAME [ASC]/DESC ; ASC IS ALSO [OPTIONAL]

ORDER OF EXECUTION

1)FROM

2)WHERE

3)GROUP BY

4)HAVING

5)SELECT

6)ORDER BY

CHARACTERISTICS-

- ORDER BY CLAUSE WILL ALWAYS EXECUTE AT LAST.

- Default order in ORDER BY clause id **ascending**
- ASC keyword is optional to pass.
- To Arrange the records in descending order we will use the DESC keyword.
- We can pass multiple columns in ORDER BY caluse

```
SQL> --WAQTD SAL OF AN EMPLOYEE IN ASCENDING ORDER.

SQL> SELECT SAL

  2  FROM EMP

  3  ORDER BY SAL ;

SQL> --WAQTD SAL OF AN EMPLOYEE IN DESCENDING ORDER–

SQL>

1  SELECT SAL

  2  FROM EMP

  3* ORDER BY SAL DESC


SQL> WAQTD ENAME ,JOB,SAL OF AN EMPLOYEE ARRANGE IT

ACCORDING TO JOB AND SAL–
```

```
SQL> SELECT ENAME,JOB,SAL
  2  FROM EMP
  3  ORDER BY JOB,SAL ;
```

7)SubQuery Operator(ALL,ANY,EXISTS,NOT EXISTS) ->

- **A** query written inside an another query is call sub query

Execution PROCESS-

INNER QUERY—------>O/P—---------->I/P OUTER QUERY—--------->O/P

1)INNER QUERY WILL EXECUTE AND GIVES US SOME OUTPUT

2)THIS OUTPUT OF INNER QUERY IS GIVEN AS A INPUT TO OUTER QUERY

3)OUTER QUERY WILL EXECUTE AND GIVE A FINAL RESULT.

CASE 1-

Whenever an **unknown is present in Ques** then we will use

Query case 1.

WAQTD NAME OF AN EMPLOYEE WHO ARE EARNING MORE

THAN ALLEN ?

```
  1  SELECT ENAME

  2  FROM EMP

  3  WHERE SAL>(SELECT SAL

  4  FROM EMP

  5* WHERE ENAME='ALLEN');
```

WAQTD DETAILS OF AN EMPLOYEE WHO ARE EARNING LESS

THAN MILLER ?

```
SQL> SELECT *

  2  FROM EMP

  3  WHERE SAL < (SELECT SAL

  4  FROM EMP

  5  WHERE ENAME ='MILLER' );
```

WAQTD DETAILS OF AN EMPLOYEE WHO IS WORKING IS SAME

DEPT AS WARD?

```
SQL> SELECT *

  2  FROM EMP

  3  WHERE DEPTNO=(SELECT DEPTNO
```

```
  4  FROM EMP

  5  WHERE ENAME='WARD');
```

WAQTD NAME ,JOB,HIREDATE OF AN EMPLOYEE IF THAT

EMPLOYEE IS WORKING IS SAME JOB AS ‘SCOTT’ JOB

```
SQL> SELECT ENAME,JOB,HIREDATE

  2  FROM EMP

  3  WHERE JOB=(SELECT JOB

  4  FROM EMP

  5  WHERE ENAME='SCOTT') ;
```

WAQTD DETAILS OF AN EMPLOYEE WHO ARE HIRED AFTER

‘BLAKE’

```
 1  SELECT *

 2  FROM EMP

 3  WHERE HIREDATE >(SELECT HIREDATE

 4  FROM EMP
```

```
 5* WHERE ENAME='BLAKE')
```

CASE 2-

WHENEVER data to be selected is present in one table and condition

to be applied is present on another table then we can use subquery

case2.

WAQTD DEPT NAME OF EMPLOYEE ADAM ?

```
SQL> SELECT DNAME
  2  FROM DEPT
  3  WHERE DEPTNO=(SELECT DEPTNO
  4  FROM EMP
  5  WHERE ENAME='ADAM');
```

WAQTD ENAME,SAL,HIREDATE,DEPTNO OF AN EMPLOYEE WHO

ARE WORKING IN NEW YORK?

```
  1  SELECT ENAME,SAL,HIREDATE,DEPTNO
  2  FROM EMP
  3  WHERE DEPTNO=(SELECT DEPTNO
```

```
  4  FROM DEPT
  5* WHERE LOC='NEW YORK')
```

-WAQTD ALL THE DEPARTMENT DETAILS OF AN EMPLOYEE IF THE EMPLOYEE IS WORKING AS A MANAGER AND EARNING MORE THAN 3000 AND HIRED AFTER 81 AND BEFORE 87–

```
  1  SELECT *
  2  FROM DEPT
  3  WHERE DEPTNO=(SELECT DEPTNO
  4  FROM EMP
  5* WHERE JOB='MANAGER' AND SAL>3000 AND HIREDATE
BETWEEN '01-JAN-82' AND '31-DEC-86' )
```

WAQTD NO OF EMPLOYEE WORKING AS A CLERK IN THE SAME DEPARTMENT AS SMITH AND EARNING MORE THAN KING AND HIRED AFTER MARTIN AND LOC IN BOSTON?

```
SQL> SELECT COUNT(*)
  2  FROM EMP
  3  WHERE JOB='CLERK' AND DEPTNO=(SELECT DEPTNO
  4  FROM EMP
```

```
  5  WHERE ENAME='SMITH') AND SAL>(SELECT SAL

  6  FROM EMP

  7  WHERE ENAME='KING') AND HIREDATE>(SELECT HIREDATE

  8  FROM EMP

  9  WHERE ENAME='MARTIN') AND DEPTNO=(SELECT DEPTNO

 10  FROM DEPT

 11  WHERE LOC='BOSTON');

SQL> --WAQTD MAXIMUM SAL OF AN EMPLOYEE

SQL> --WAQTD NAME AND SAL OF AN EMPLOYEE WHO ARE

EARNING MAX SAL

SQL> SELECT ENAME,SAL

  2  FROM EMP

  3  WHERE SAL=(SELECT MAX(SAL)

  4  FROM EMP);
```

04/09/24

WAQTD DNAME OF ALLEN?

WAQTD DNAME OF ADAM?

WAQTD DNAME OF MILLER AND WARD?

**** Types of subquery

1.Single row subquery

- A subquery returning only one value /output is called a single row subquery , this output of a single row subquery is given as an input to the output query.
- Both **=** and **In** operator can be used to handle single output of single row subquery.

2)multi row subquery

- The subquery returning multiple values/output is known as a multi row subquery.
- We can use only In operator in multi row subquery.

SUB QUERY OPERATOR

1)ALL OPERATOR

- All operator will return true if all the values on the right hand side are satisfying the condition .
- All operator will always give along with relational operator.

E.g - >All

<All

>=All

<=All

- All operator will work like an And operator.

2)Any OPERATOR

- Any operator will be Written true if any one of values on the right hand side is satisfying the condition.

NOTE-.This Any operator is also used along with relational operators.

Eg. >Any

<Any

>=Any

<=Any

NOTE- Any operator will work like an **OR operator.**

```
SELECT ENAME,SAL

FROM EMP

WHERE SAL>ALL(SELECT SAL

                FROM EMP

                WHERE DEPT=10);
```

WAQTD NAME AND SAL OF AN EMPLOYEE IF ANY EMPLOYEE IS

EARNING MORE THAN ANY ONE EMPLOYEE OF DEPARTMENT

10?

```
SQL> SELECT ENAME,SAL
  2  FROM EMP
  3  WHERE SAL>ANY(SELECT SAL
  4  FROM EMP
  5  WHERE DEPTNO=10);
```

WAQTD NAME OF AN EMPLOYEE IF THE EMPLOYEE EARN LESS

THAN THE EMPLOYEE WORKING AS SALESMAN?

WAQTD NAME OF AN EMPLOYEE IF THE EMPLOYEE EARN LESS

THAN THE ATLESAT ONE EMPLOYEE WHO IS WORKING AS

SALESMAN?

```
1  SELECT ENAME
2  FROM EMP
3  WHERE SAL<ANY(SELECT SAL
4  FROM EMP
5* WHERE JOB='SALESMAN')
```

WAQTD MAX SAL OF AN EMPLOYEE

3)NESTED SUBQUERY

- A subquery Written inside another subquery is called a nested subquery.
- Limit–255 we can write a maximum 255 sub query.

WAQTD 2ND MAX SALARY?

```
SQL> SELECT MAX(SAL)
  2  FROM EMP
  3  WHERE SAL<(SELECT MAX(SAL)
  4  FROM EMP);
```

6/09/24

Employee Manager Relationship

WAQTD NAME OF SMITH'S MANAGER?

```
SQL> SELECT ENAME
  2  FROM EMP
  3  WHERE EMPNO IN(SELECT MGR
  4                 FROM EMP
  5                 WHERE ENAME='SMITH');
```

WAQTD DETAILS OF BLAKE'S MANAGER?

WAQTD MANAGER DETAILS OF CORD?

WAQTD DEPARTMENT NAME AND LOC OF MANAGER OF CLARK?

WAQTD DETAILS OF SMITH'S MANAGER'S MANAGER?

WAQTD DEPARTMENT DETAILS OF SCOTT'S MANAGER'S

MANAGER?

******NOTE-> IF we find Manager details we compare**

MGR=EMPNO

******NOTE-> IF we find Employee details we compare

EMPNO=MGR

WAQTD OF EMPLOYEE WORKING UNDER BLAKE?

```
SELECT *
FROM EMP
WHERE MGR=(SELECT EMPNO
  FROM EMP
WHERE ENAME='BLAKE');
```

WAQTD DETAILS OF EMPLOYEES WHO ARE WORKING UNDER

KING?

WAQTD DEPARTMENT DETAILS OF AN EMPLOYEE WHO ARE

REPORTING TO JONES?

Drawbacks of Sub query

- Data from multiple tables can't be displayed simultaneously on the output screen from multiple tables.
- As conditions are increased the length of sub query is also increased.

- These two drawbacks can be solved with the help of joins.

JOINS

- To retrieve the data from multiple tables.

Type of join

1) **Cross Join/ cartesian Join**
2) **Inner Join**
3) **Natural Join**
4) **Outer Join—>{ left Outer Join, Right Outer Join, Full Outer Join}**
5) **Self Join**

1)**Croshttps://docs.google.com/document/d/1TWn4qvsYXKFBOtffgwbYLnfNblJX-dgrxzPubbunKLo/edit?usp=drive_links Join/ cartesian Join -**

It is mainly use **to merge all the Records** of tables 1 with all the records of table 2

Syntax-

ANSI(American national standard institute)

SELECT COLNAME/EXPRESSION

FROM TABLENAME1 CROSS JOIN TABLENAME2 ;

ORACLE

SELECT COLNAME,EXPRESSION

FROM TABLENAME1,TABLENAME2 ;

EMP TABLE

EMPNO	ENAME	DEPTNO
1	SCOTT	20
2	BLAKE	30
3	MILLER	10

DEPT TABLE

DEPTNO	DNAME	LOC
10	ACCOUNTING	M
20	SALES	G

30 RESEARCH B

SELECT *

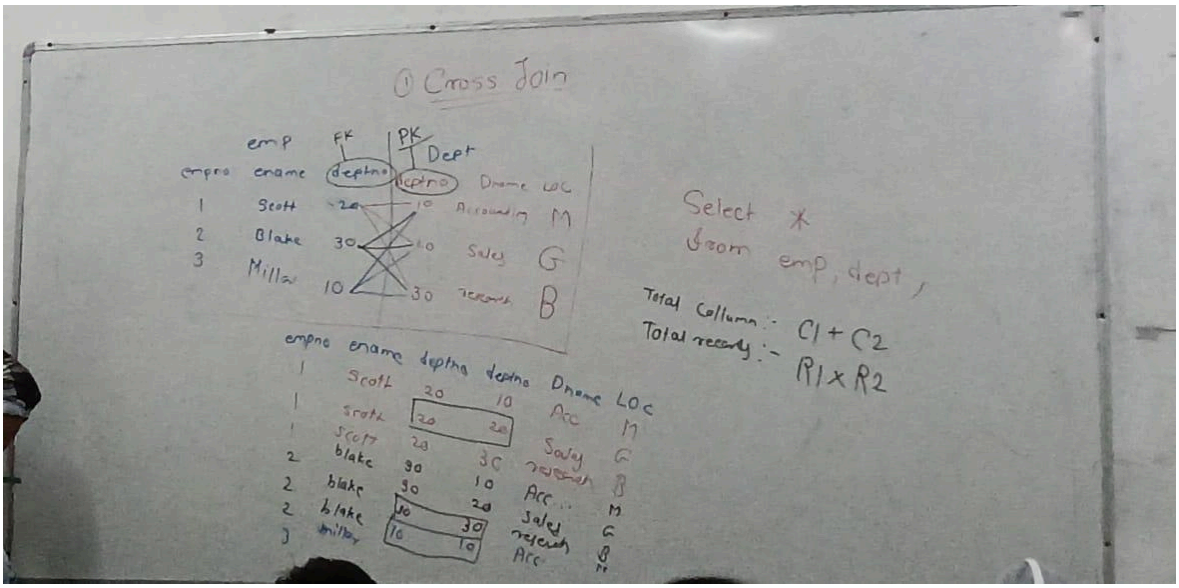
FROM EMP,DEPT ;

EMPNO ENAME DEPTNO DEPTNO DNAME LOC

1 SCOTT 20 10 ACC M

1 SCOTT 20 20 SALES G

1SCOTT 20 20 SALES G



DRAWBACK of cross Joins

- No of unmatched records are more than the no. of matching records.

2.**Inner Join-**

It is mainly used to obtain matching records from two or more tables.

Syntax

ANSI(American national standard institute)

SELECT COLNAME/EXPRESSION

FROM TABLENAME1 INNER JOIN TABLENAME2

ON <JOIN CONDITION>;

ORACLE

SELECT COLNAME,EXPRESSION

FROM TABLENAME1,TABLENAME2

WHERE <JOIN CONDITION>;

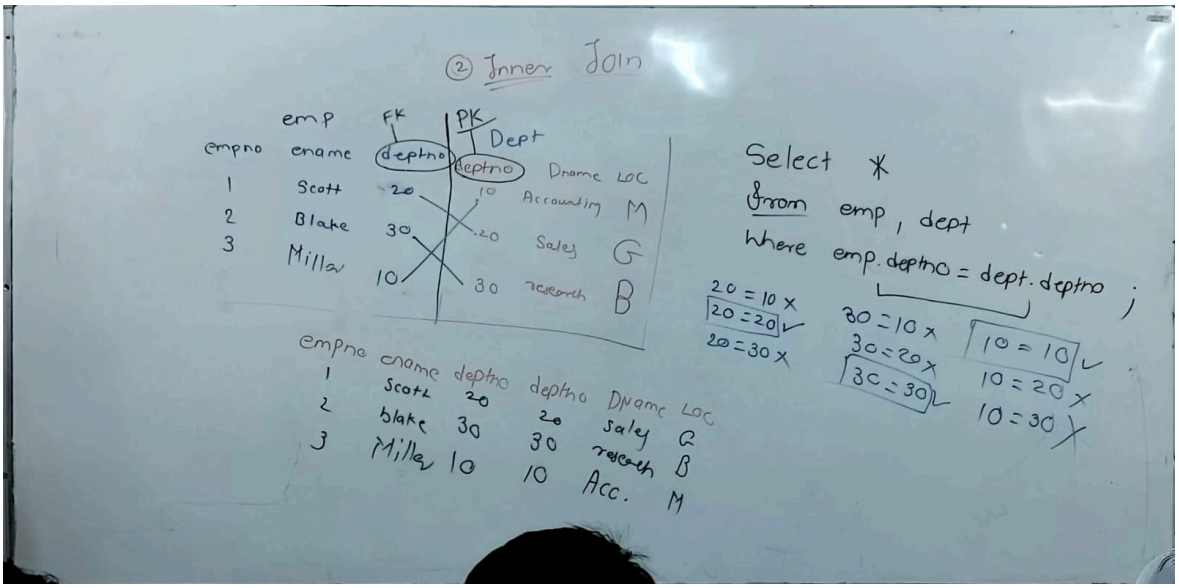
JOIN CONDITION-

TABLENAME1.COMMON COLUMN=TABLE NAME 2.COMMON

COLUMN

E.X

EMP.DEPTNO=DEPT.DEPTNO



WAQTD EMPNAME AND DNAME OF ALL THE EMPLOYEE

```
SQL> SELECT ENAME,DNAME
  2  FROM EMP,DEPT
  3  WHERE EMP.DEPTNO=DEPT.DEPTNO;
```

WAQTD ENAME AND LOC OF ALL THE EMPLOYEE

```
SQL> SELECT ENAME,LOC
  2  FROM EMP,DEPT
  3  WHERE EMP.DEPTNO=DEPT.DEPTNO AND JOB='ANALYST' ;
```

***WAQTD DEPTNO AND ENAME AND LOC OF ALL THE**

EMPLOYEE

SELECT EMP.DEPTNO,DEPT.DEPTNO,ENAME,LOC

FROM EMP,DEPT

WHERE EMP.DEPTNO=DEPT.DEPTNO

WAQTD ENAME,SAL,DNAME OF ALL THE EMPLOYEE WORKING

AS A CLERK IN DEPT 20 WITH A SALARY MORE THAN 1800?

WAQTD ENAME ,DNAME,LOC OF AN EMPLOYEE IF EMPLOYEE IS

EARNING MOARE THAN 2000 IN NEW YORK ?

3) Natural Join-

Natural join will behave like an inner join if there is a relation

between two or more tables.

Else it behave like a cross join.

Sintax-

ANSI(American national standard institute)

SELECT COLNAME/EXPRESSION

FROM TABLENAME 1 NATURAL JOIN TABLENAME 2;

WAQTD ALL THE DETAILS FROM EMP AND DEPT?

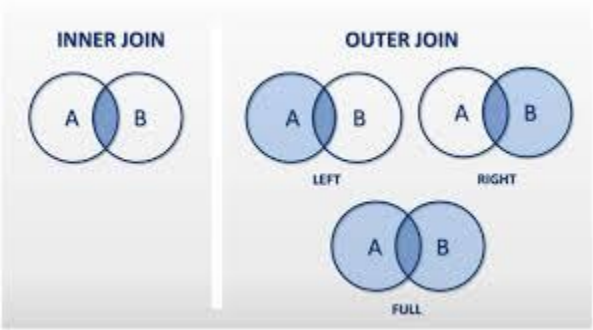
SELECT *

FROM EMP NATURAL JOIN DEPT ;

4) Outer Join-

It is used to obtain the unmatch record ,Along with that we will get

matching records.



It is classified into three 3 types

a)left outer join

It is used to obtain unmatched records from the left table

and matching records from both the tables.

Syntax-

ANSI(American national standard institute)

SELECT COLNAME/EXPRESSION

FROM TABLENAME1 LEFT[OUTER] JOIN TABLENAME2

ON <JOIN CONDITION> ;

ORACLE

SELECT COLNAME/EXPRESSION

FROM TABLENAME1 ,TABLENAME2

WHERE <JOIN CONDITION>

JOIN CONDITION-> EMP.DEPTNO=DEPT.DEPTNO (+) ;

SELECT *

FROM EMP ,DEPT

Right Outer Join-

It is used to obtain unmatched records from the right table and matching records from both the tables.

Syntax-

ANSI

SELECT COLNAME/EXPRESSION

FROM TABLENAME1 RIGHT[OUTER] JOIN TABLENAME2

ON <JOIN CONDITION> ;

ORACLE

SELECT COLNAME/EXPRESSION

FROM TABLENAME1 ,TABLENAME2

WHERE <JOIN CONDITION>

JOIN CONDITION—> EMP.DEPTNO(+)=DEPT.DEPTNO ;

SELECT *

FROM EMP,DEPT

WHERE EMP.DEPTNO(+)=DEPT.DEPTNO

WAQTD LOC OF EMPLOYEE WHO ARE EARNING COMM ?

SELECT LOC

FROM EMP,DEPT

WHERE COMM IS NOT NULL

WAQTD ENAME AND DNAME IF NO EMPLOYEE IS WORKING IN

ANY DEPT?

C)FULL OUTER JOIN-

It is used to obtain unmatched records from the right

table,left table and matching records from both the tables.

Syntax-

```
SELECT COLNAME/EXPRESSION

FROM TABLENAME1 FULL[OUTER] JOIN  TABLENAME2

ON <JOIN CONDITION> ;
```

```
SELECT *

 FROM EMP FULL JOIN DEPT
```

ON EMP.DEPTNO=DEPT.DEPTNO ;

WAQTD EANAME AND DNAME OF EMPLOYEE EVENTHO NO

EMPLOYEE IS WORKING IN ANY DEPARTMENT?

```
SELECT ENAME,DNAME
FROM EMP,DEPT
WHERE EMP.DEPTNO=DEPT.DEPTNO(+)
```

SELF JOIN-

It is mainly used to join the table with itself .(SAME TABLE)

Syntax

ANSI

```
SELECT COLNAME/EXPRESSION
FROM TABLENAME1 [SELF] JOIN TABLENAME1
ON <JOIN CONDITION>;
```

ORACLE

SELECT COLNAME/EXPRESSION

FROM TABLENAME1,TABLENAME1

WHERE <JOIN CONDITION> ;

Query1-WAQTD EMPLOYEE NAME AND MANAGER NAME OF

SCOTT?

```
SELECT E.ENAME,M.ENAME
FROM EMP E,EMP M
WHERE E.MGR=M.EMPNO AND M.ENAME='SCOTT' ;
```

Query2-WAQTD DETAILS OF EMPLOYEE AND MANAGER OF

SMITH'S?

```
SELECT E.*,M.*
FROM EMP E,EMP M
WHERE E.MGR=M.EMPNO AND E.ENAME='SMITH
```

Query3-WAQTD EMPLOYEE NAME AND MANAGER NAME AND

THEIR DEPARTMENT NO IF EMPLOYEE IS WORKING AS CLERK?

```
SELECT E.ENAME ENAME,M.ENAME MANAGER
```

FROM EMP E,EMP M

WHERE E.MGR=M.EMPNO AND E.JOB='CLERK' ;

Query4-WAQTD ENAME,MANAGER'S JOB OF MANAGER WORKS

AS ANALYST?

```
SELECT E.ENAME ENAME,M.JOB JOB
FROM EMP E,EMP M
WHERE E.MGR=M.EMPNO AND M.JOB='ANALYST'
```

Query5- WAQTD ENAME AND MANAGER'S NAME ALONG WITH

THEIR JOB IF EMP AND MANAGER ARE WORKING FOR SAME

DESIGNATION?

```
SELECT E.ENAME ENAME,E.JOB JOB,M.ENAME
MANAGER,M.JOB JOB
FROM EMP E,EMP M
WHERE E.MGR=M.EMPNO AND E.JOB=M.JOB
```

Query6- WAQTD ENAME EMP SALARY MANAGER'S NAME

MANAGER’S SALARY IF MANAGER EARNS MORE THAN

EMPLOYEE?

```
SELECT E.ENAME ENAME,E.SAL ESAL,M.ENAME MNAME,M.SAL
MSAL
FROM EMP E,EMP M
WHERE E.MGR=M.EMPNO AND M.SAL>E.SAL ;
```

Query7-WAQTD DETAILS OF SMITH’S MANAGER’S?

```
SELECT S.ENAME,SM.ENAME,SMM.ENAME
FROM EMP S,EMP SM,EMP SMM
WHERE S.MGR=SM.EMPNO AND SM.MGR=SMM.EMPNO AND
S.ENAME='SMITH' ;
```

Query8- WAQTD MANAGER NAME AND DEPT NAME OF SMITH’S

Query9-WAQTD EMPLOYEE NAME MANAGER NAME AND HIS

MANAGER’S NAME OF ALLEN ALONG WITH THAT DISPLAY THE

DEPARTMENT NAME OF ALLEN’S MANAGER’S MANAGER?

```
SELECT A.ENAME,AM.ENAME,AMM.ENAME,D.DNAME

FROM EMP A,EMP AM,EMP AMM,DEPT D

WHERE A.MGR=AM.EMPNO AND AM.MGR=AMM.EMPNO AND

AMM.DEPTNO=D.DEPTNO
```

Query10-WAQTD NUMBER OF EMPLOYEE REPORTING TO

BLAKE'S MANAGER AND EARNING SAL LESS THAN FORD'S

MANAGER?

```
 SELECT COUNT(*)

 FROM EMP E,EMP B,EMP BM,EMP F,EMP FM

 WHERE B.MGR=BM.EMPNO AND F.MGR=FM.EMPNO AND

E.SAL<FM.SAL AND B.ENAME='BLAKE' AND F.ENAME='FORD'
```

Query11-WAQTD TOTAL SAL REQ TO PAY ALL THE EMPLOYEE

WORKING IN SAME LOCATION AS SCOTT'S MANAGER?

```
 SELECT SUM(E.SAL)

FROM EMP E,DEPT D1,EMP S,EMP SM,DEPT D2

WHERE E.DEPTNO=D1.DEPTNO AND S.MGR=SM.EMPNO AND

SM.DEPTNO=D2.DEPTNO AND D1.LOC=D2.LOC AND
```

S.ENAME='SCOTT' ;

Query12-WAQTD NAME ,JOB,DEPTNAME,LOC OF ALL THE

EMPLOYEE WORKING IN SAME DEPARTMENT AS THAT OF

SMITH’S MANAGER ‘S MANAGER?

```
 SELECT E.ENAME,E.JOB,D1.DNAME
 FROM EMP E,DEPT D1,EMP S,EMP SM,EMP SMM,DEPT D2
 WHERE E.DEPTNO=D1.DEPTNO AND S.MGR=SM.EMPNO AND
SM.MGR=SMM.EMPNO AND SMM.DEPTNO=D2.DEPTNO AND
D1.DEPTNO=D2.DEPTNO AND S.ENAME='SMITH'
```

Query13- WAQTD EMPLOYEE NAME AND HIS DEPARTMENT

NAME ALONG WITH MANAGER NAME AND MANAGER

DEPARTMENT NAME IF MANAGER IS EARNING GREATER THAN

THE EMPLOYEE AND COMMISION OF EMPLOYEE SHOULD BE

NULL AND EMPLOYEE SHOULD BE HIRED AFTER THE

MANAGER?

```
 SELECT E.ENAME,D1.DNAME EDNAME,EM.ENAME
MNAME,D2.DNAME MDNAME
```

FROM EMP E,DEPT D1,EMP EM,DEPT D2

WHERE E.DEPTNO=D1.DEPTNO AND E.MGR=EM.EMPNO AND

EM.DEPTNO=D2.DEPTNO AND EM.SAL>E.SAL AND E.COMM IS

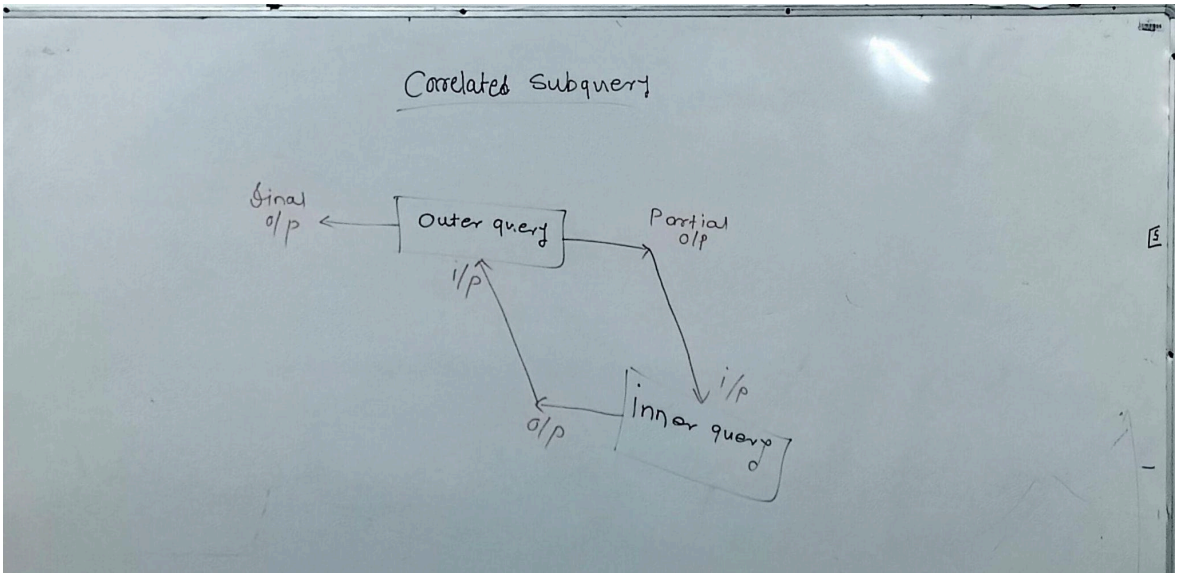
NULL AND E.HIREDATE<EM.HIREDATE

CORE RELATED SUB QUERY

COMBINATION OF JOINS AND SUBQUERY

Core related sub query in which both outer query and inner query are

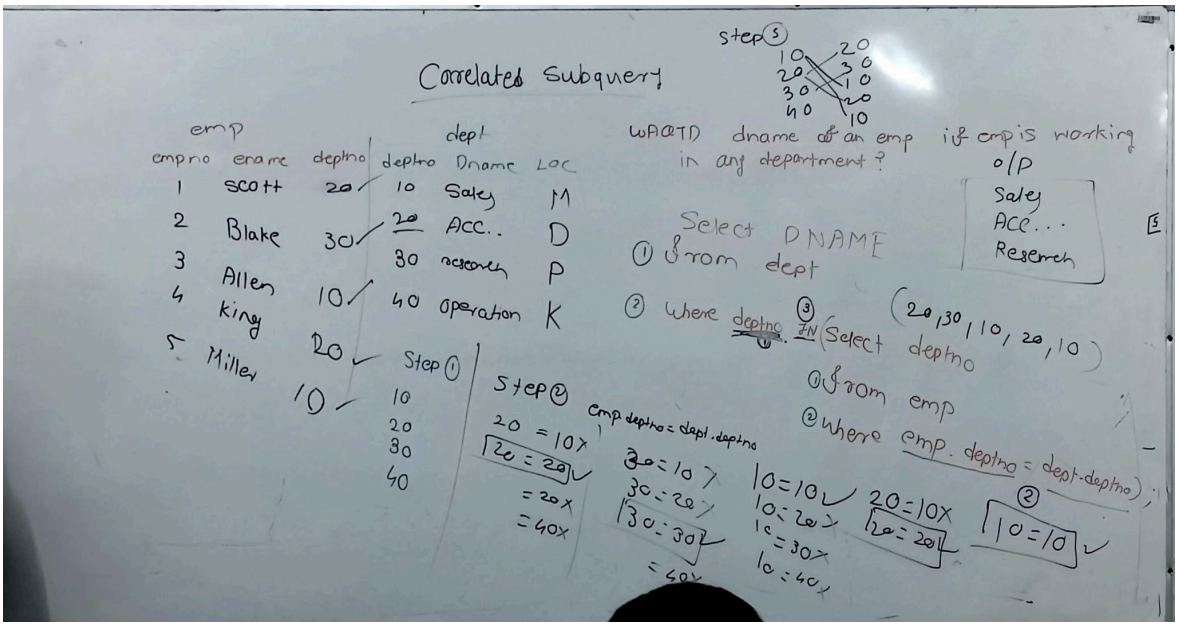
interdependent on each other .



Execution Process of CORRELATED SUBQUERY

- First outer query will execute partially and give us partial output.
- This output of outer query is given as a input to the inner query

- Innerquery will execute and give us an output.
- This output of inner query is given as a input to outer query
- At last the outer query will fully execute and give us a final output.



Exists Operator

Exist Operator is a urinary operator.

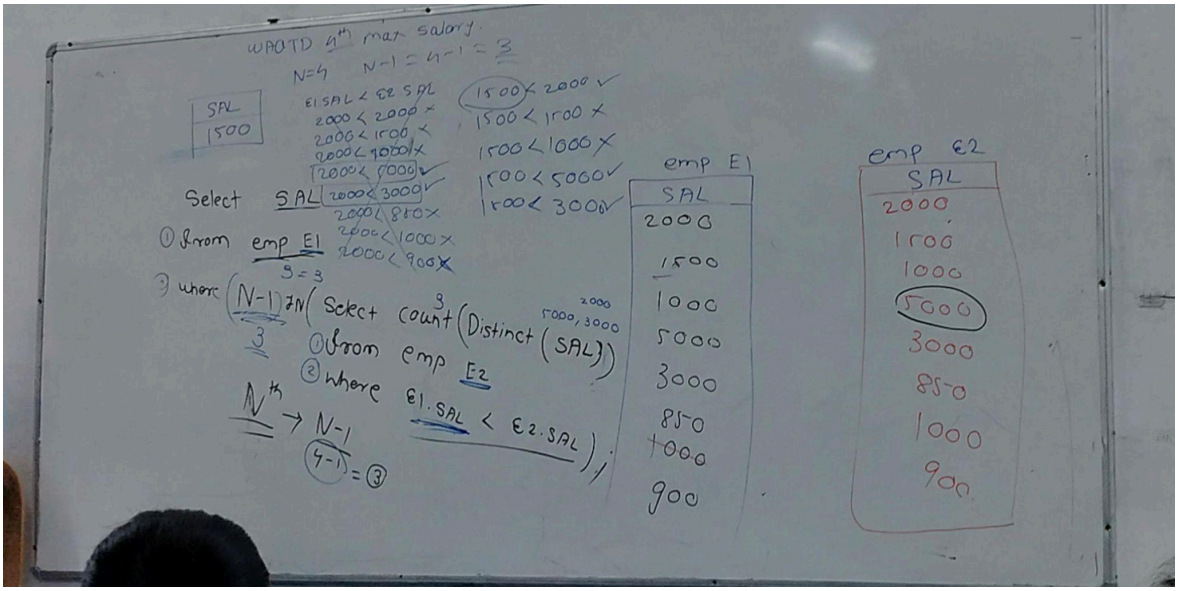
Exist Operator returns true , if subquery is returning any value other than null.

WAQTD DNAME OF A EMPLOYEE IF EMPLOYEE IS NOT

WORKING IN ANY DEPATMENT?

```
SELECT DNAME

FROM DEPT

WHERE NOT EXISTS(SELECT DEPTNO

                 FROM EMP

                 WHERE EMP.DEPTNO=DEPT.DEPTNO )
```

WAQTD NTH MAX AND NTH MIN SAL?



WAQTD 7TH MIN SAL OF AN EMPLOYEE?

```
SELECT SAL
```

```
FROM EMP E1

WHERE 6 IN (SELECT COUNT(DISTINCT(SAL))

        FROM EMP E2

      WHERE E1.SAL>E2.SAL)
```

WAQTD 3RD ,4TH,7TH,8TH,9TH MAX SAL?

```
SELECT DISTINCT (SAL)

FROM EMP E1

WHERE (SELECT COUNT(DISTINCT(SAL))

        FROM EMP E2

      WHERE E1.SAL<E2.SAL) IN (2,3,6,7,8)
```

IN MULTIPLE SAL WE WILL INTERCHANGE THE CONDITION

SINGLE ROW FUNCTION

Is a function which accepts multiple inputs and gives us multiple respective outputs.

Types of SRF

1) Length()
2) Concat()

3) Lower()

4) Upper()

5) INITCAP()

6) REVERSE()

7) SUBSTR()

8) INSTR()

9) MOD()

10) ROUND()

11) TRUNC()

12) MONTHS-BETWEEN()

13) LAST-DAY()

14) TO-CHAR()

15) NVL()

1)LENGTH()

- It is mainly used to count the total no. of characters present inside a particular string.
- Syntax:

```
length('string')
```

WAQTD THE TOTAL NO. OF CHARACTER PRESENT INSIDE

AN EMPLOYEE NAME?

```
SELECT LENGTH(ENAME)
FROM EMP;
```

WAQTD NAME OF THE EMPLOYEE WHICH CONSIST OF

TOTAL 6TH CHARACTERS?

```
SELECT ENAME
FROM EMP
WHERE LENGTH(ENAME)=6 ;
```

WAQTD SAL OF AN EMPLOYEE WHO ARE EARNING TOTAL

4 DIGIT OF SAL?

```
SELECT SAL
FROM EMP
WHERE LENGTH(SAL)=4
```

2.Concat():

- It is mainly used to concat two strings.
- Syntax:

  concat('str1','str2')

WAQTD THE STRING IN A PARTICULAR FORMAT BY

CONCATENATING ‘MR’ AND ‘ANKIT’ ?

SELECT CONCAT(‘MR’,’ANKIT’)

FROM DUAL ;

WAQTD THE STRING LIKE ‘MR SCOTT’ FOR EACH EMPLOYEE ?

```
SELECT CONCAT('MR ',ENAME)

FROM EMP ;
```

WAQTD THE STRING LIKE ‘HI I AM SMITH’ FOR EACH

EMPLOYEE?

```
SELECT CONCAT('HI I AM ',ENAME)

FROM EMP;
```

3)LOWER():

- It is used to convert a string into LOWERCASE.
- Syntax:

LOWER(‘STR’)

WAQTD ALL THE NAMES OF AN EMPLOYEE IN LOWERCASE?

```
SELECT LOWER(ENAME)

FROM EMP;
```

4)UPPER():

- It is mainly used to convert a string in UPPERCASE.
- Syntax:

  UPPER('STR')

5) INITCAP():

- It is mainly used to convert a string in which the first letter will be in capital case and remaining letter in smaller case.
- Syntax:

  INITCAP('STR')

WAQTD NAME OF EMPLOYEE IN WHICH FIRST CHARACTER IS IN UPPERCASE AND REMAINING IN LOWER CASE.

```
SELECT INITCAP(ENAME)
FROM EMP;
```

6) REVERSE():

- It is mainly used to reverse a string.
- Syntax:

  REVERSE('STR')

WAQTD ENAME OF EMPLOYEE IN REVERSE ORDER?

```
SELECT REVERSE(ENAME)

FROM EMP;
```

7) SUBSTR():

- ❖ It is mainly used to extract substring from the original string.
- ❖ Syntax:

SUBSTR('Original string', start_position, length)

start_position: The position where the extraction should begin

(1-based index).

length: (Optional) The number of characters to extract.

WAQTD SPIDERS FROM THE STRING QSPIDERS?

```
SELECT SUBSTR('QSPIDERS',2)

FROM DUAL
```

WAQTD FIRST THREE CHARACTERS OF A STRING QSPIDERS?

```
SELECT SUBSTR('QSPIDERS',2)

FROM DUAL
```

WAQTD FIRST TWO CHARACTER OF ENAME?

```
SELECT SUBSTR(ENAME,1,3)
FROM EMP ;
```

WAQTD LAST THREE CHARACTER OF EMPLOYEE NAME?

```
SELECT SUBSTR(ENAME,-3)
FROM EMP;
```

WAQTD NAME OF THE EMPLOYEE WHOSE JOB CONSIST OF

FIRST THREE CHARACTER AS MAN?

```
SELECT ENAME
FROM EMP
WHERE SUBSTR(JOB,-3)='MAN'
```

WAQTD FIRST HALF OF ENAME?

```
SELECT ENAME,SUBSTR(ENAME,1,LENGTH(ENAME)/2)
FROM EMP;
```

WAQTD THE SECOND HALF OF ENAME?

```
SELECT ENAME,SUBSTR(ENAME,LENGTH(ENAME)/2+1)
FROM EMP;
```

8) INSTR():

- The `INSTR()` Is used to **find the position of a substring within a string**.
- If SUB_string is present in the original string then it will give position otherwise we will get 0.
- Syntax:

  INSTR('original str' , 'SUB_str' , position [occurrence])

WAQTD name of an employee consist of a present?

WAQTD ENAME if it consists of at least 2 'a'?

9) Replace():

- Replace is mainly used to replace a given string with new string in a original string.
- Syntax:

  REPLACE('ORIGINAL_string', 'old_string', 'new_string')

Ex. replace ('BANANA' ,'A','C')==> BCNCNC

replace ('BANANA' ,'NA','M')==> BAMM

WAQTD string in which employee name first two characters should be

in capital case , third character will be as it is and remaining character

will be in lowercase?

```
SELECT REPLACE(REPLACE('ankIT', 'an', UPPER('an')), 'IT',
LOWER('IT'))
FROM DUAL;
```

10) MOD():

- It is used to obtain the remainder value between two values.
- Syntax:

  mod(value1,value2)

E.g

mod(111, 3)-----> 0

mod(55,4)-------->3

WAQTD even employee id of an employee?

```
SELECT EMPNO
FROM EMP
WHERE MOD(EMPNO, 2) = 0;
```

WAQTD name and sal of an employee if the employee is earning odd sal?

```
SELECT ENAME, SAL
FROM EMP
WHERE MOD(SAL, 2) = 1;
```

11) ROUND():

- It is used to obtain a round off value for a number according to the scale value.
- Syntax:

  round(value[,scale])

E.g:

round(420.56)----> 421

round(420.1234)-----> 420

round(786.453,0)-------> 786

round(786.9543,2)-----> 786.95

round(7864.943,-2)------> 7900

0 5→10

0 50—–> 100

0 500—–--> 1000

12) TRUNC():

- ❖ It is used to obtain the round off value to the smallest value for a given number.
- ❖ Syntax:

  Trunc(value[,scale]

E.g:

Trunc(420.56)----> 420

Trunc(420.1234)-----> 420

Trunc(786.453,0)-------> 786

Trunc(786.9543,2)-----> 786.95

Trunc(7864.943,-2)------> 7800

0 5→10

0 50—–> 100

0 500—––> 1000

13) MONTHS_BETWEEN():

- ❖ It is mainly used to obtain numbers of months present between two given dates.
- ❖ Syntax:

  Months_Between (Date1,Date2)

a) Sysdate

b) Current-date

c) SystimeStamp

WAQTD experience of an employee in a month?

```
SELECT MONTHS_BETWEEN(CURRENT_DATE,HIREDATE)
FROM EMP
```

WAQTD experience of an employee in a YEARS?

```
SELECT
ROUND(MONTHS_BETWEEN(CURRENT_DATE,HIREDATE)/12)
FROM EMP
```

14) TO_CHAR():

- It is mainly used to convert a date into string in a particular format model.
- Syntax:

  To_Char(Date,format model)

Format model:

YYYY-1980

YY-80

MONTH-September

MON-SEPT

MM-02

DAY-

DD-14

HH24

HH12

SS

M1

WAQTD NAME AND HIREDATE OF EMPLOYEE WHO ARE HIRE ON

WEDNESDAY ?

```
SELECT ENAME,HIREDATE

FROM EMP

WHERE TO_CHAR(HIREDATE,'DAY')='WEDNESDAY'
```

15) LAST_DAY() :

- It is mainly used to obtain the last day of a particular month.
- Syntax:

```
LAST_DAY(Date)
```

16) NVL():Null value logic

- It is mainly used to prevent or to eliminate the drawbacks of null
- Syntax:

  NVL(colName,value)

E.g

```
SELECT SAL + NVL(COMM,0)
FROM EMP ;
```

DDL(Data Definition language)

It is mainly used to create or to manage the structure.

1) Create
2) Alter
3) Rename
4) Truncate
5) Drop

1) Create

- It is mainly use create new table structure
- SYNTAX:

```
CREATE TABLE TNAME
(
COL1 DATA TYPE CONSTRAINT,
COL2 DATA TYPE CONSTRAINT);
```

E.g:

```
CREATE TABLE STUDENT
 (
 ID NUMBER(5) PRIMARY KEY CHECK (ID>0) CHECK
(LENGTH(ID)=5),
 NAME VARCHAR(20) NOT NULL ,
 COURSE VARCHAR(20) ,
 PHONE_NO NUMBER(10) CHECK (PHONE_NO>9)
CHECK(LENGTH(PHONE_NO)=10),
 GENDER VARCHAR(6),
 DOB DATE );
```

2) RENAME

- It is mainly used to change the name of the table.
- Syntax:

  Rename old-table-name To new -table-name;

3) ALTER

- It is mainly use to modify the table structure
- Syntax:

  ALTER TABLE TABLENAME

  1)To add a column.

Syntax:

  ALTER TABLE TABLENAME

  ADD COLNAME DATA TYPE CONSTRAINT ;

E.G: ALTER TABLE STUDENT

ADD ADDRESS VARCHAR(20);

NOTE :WE WILL USE DESCRIBE AND SHOW THE EMPTY TABLE

THAN WE WILL USE .

  DESC TABLENAME;

*ADD COLNAME DATA TYPE

2) To rename a column

SYNTAX: ALTER TABLE TABLENAME

RENAME COLNAME OLD-COL-NAME TO NEW_COL_NAME;

3) To drop a column

SYNTAX: ALTER TABLE TABLENAME

DROP COLUMN COLNAME;

4) To modify datatype

SYNTAX: ALTER TABLE TABLENAME

MODIFY COLNAME NEW DATA TYPE

E.G: ALTER TABLE STUDENT

MODIFY NAME VARCHAR(30) ;

5) To modify constraint

SYNTAX: ALTER TABLE TABLENAME

MODIFY COLNAME EXISTING DATA_TYPE NEW_CONSTARINT;

E.G: ALTER TABLE STUDENT

MODIFY GENDER VARCHAR(10) NOT NULL;

6) To add a constraint separately

SYNTAX: ALTER TABLE TABLENAME

ADD CONSTRAINT CONSRAINT_REFRANCE_NAME

E.G: ALTER TABLE STUDENT

ADD CONSTRAINT UK_PHONE_NO UNIQUE(PHONE_NO);

7) To add a foreign key.

While creating a table

Syntax:

E.g:

CREATE TABLE TRAINER

(

T_ID NUMBER(5) PRIMARY KEY CHECK (T_ID>0)

CHECK(LENGTH(T_ID)=5),

T_NAME VARCHAR(10),

COURSE VARCHAR(10),

ID NUMBER(5),

CONSTRAINT FK_ID FOREIGN KEY(ID) REFERENCES

STUDENT(ID));

2ND WAY

To assign a foreign key in already create a table.

Syntax:

ALTER TABLE TABLE1

ADD CONSTRAINT FK_ID FOREIGN KEY(::) REFERENCE

TABLE2(ID)

To drop a constraint.

Syntax: Drop Constraint Constraint_ref_Name

E.g:

Alter Table Trainer

Drop Constraint FK_ID;

** To add foreign key in already exist table

Syntax: Alter Table TableName1

Add Id number(5),

Add Constraint FK_Id foreign key referenceTableName2(Id);

4) Truncate

It is mainly used to delete the data from the table without affecting table structure.

Syntax: Truncate Table TableName;

5) Drop

It is mainly used to delete the table (It will delete the data as well as table structure)

Syntax: Drop Table TableName;

When we drop a table then the table will be stored in the recycle bin .

If we want to delete the table permanently .

Syntax: purge Table TableName;

If we want to recycle table as well as table data.

Syntax: Flashback Table TableName

To before Drop;

DML(Data manipulation language)

It is used to deal with the data .

We can insert , delete and update the data.

1) **Insert:**

To insert all data in a table

Syntax: Insert Into TableName values(v1,v2,v3,...vn)

To insert data in a particular column

Syntax:

Insert Into TableName (col1,col2…coln) values (v1,v2,vn)

To insert data through user input

Syntax:

Insert Into TableName

values(&ID,'&Name','&Course','&phone_no','&gender','&DOB');

Insert Into TableName(ID,Name,Course)

values(&id,'&Name','&Course');

**To insert multiple record in table

```
Insert All

into Student values(v1,v2,...)

into Student values(            )

into Student values(            )

Select * from DUAL;
```

2) Update:

Update is mainly used to modify the existing record.

Syntax:

```
Update TableName

Set col1=v1, col2-v2,Where <condition>;
```

WAQTD THE JOB OF MILLER AS DEVELOPER?

```
UPDATE EMP

SET JOB=DEVELOPER

WHERE ENAME ='MILLER';
```

3) Delete:

Delete is mainly used to delete a particular record.

Syntax: Delete from TableName;

Where<condition>;

WAQTD THE DATA OF BLAKE?

DELETE FROM EMP

WHERE ENAME='BLAKE';

TCL (Transition Control language)

It is mainly used to save the transition .

1)Commit

2)Save point

3)Rollback

1)Commit:

It is mainly used to save all the transitions permanently.

Syntax:

Commit;

2) Save point:

It is mainly used to save the location.

Syntax:

Savepoint SavepointName[any name];

3) Rollback:

It is mainly use to reach the latest saved location.

Syntax:

Rollback To Savepoint name;

Insert

Insert

Insert

Commit;

Insert

Insert

Savepoint S1;

Insert

Insert

Savepoint S2;

Insert

Insert

Rollback To S2;

Rollback To S1;

DCL(Data Control language)

It is mainly used to control the data flow between the users.

1)Grant

2)Revoke

1)Grant:

It is mainly use to give an access of data to a user.

Syntax:

Grant Command_Name

On Table_Name

To User_Name;

2) Revoke:

It is mainly use to take back the access on data from the user.

Syntax:

Revoke Command_Name

On Table_Name

From User_Name;

Key Attribute/Candidate Key:

The key attribute is an attribute which can identify the another attributes uniquely is called as key attribute.

Student table

ID	key attribute/Candidate Key
Name	ID
Age	Aadhar
DOB	mail
Phone_no	pan
Mail	
Gender	

Add

Acc_no

Non-key attributes:

All the attributes other than the key attributes .

Non-key attribute

Name

Age

Course

DOB

Gender

Add

Prime-key attributes:

Among all the key attributes any one key attribute is chosen to identify

another attribute uniquely.

Ex. ID

Non Prime key attribute:

The key attribute other than the prime key attribute.

E.g: Aadhar,phone_no, pan,acc_no

Composite key attribute:

The combination of more than two non key attribute which can identify

an attribute uniquely is called as composite key attribute.

E.g; Name,Age,DOB

Super key attribute:

The set of all key attributes is called as super key attribute.

E.g: id,aadhar, phone_no,mail,pan,acc_no

Foreign Key attribute:

An attribute which behaves as a part of an another entity is called

foreign key attribute.